Casino_Challenge_MAB_Workshop

June 26, 2026

1 Multi-Armed Bandit Workshop Submission

Name: Ali Cihan Ozdemir **Course:** CSCN8020

1.0.1 Introduction

- **Objective:** Navigate the exploration-exploitation trade-off using ϵ-greedy strategies.
- **Round 1:** Stationary environment (fixed true means). Addressed using a decaying ϵ schedule to lock into the optimal arm.
- **Round 2:** Non-Stationary environment (drifting means). Addressed using a constant step-size (α) to track moving targets.

2 The Casino Challenge — Multi-Armed Bandits & -Greedy (Gamified Workshop)

Goal: Compete to maximize reward while learning the exploration–exploitation trade-off using **-greedy** policies.

You will: - Implement and *play* with -greedy on a fixed set of bandit arms (Round 1: Stationary). - Compete on a leaderboard (submit your score locally). - Reflect on how exploration affects performance. - Face a twist (Round 2: **Non-Stationary** bandits) and adapt your strategy.

2.1 Setup

Run the cell below. If you're on Colab/Jupyter, required libraries should already be available.

[14]:
```
import numpy as np
import matplotlib.pyplot as plt
import time
from pathlib import Path
from datetime import datetime

plt.rcParams['figure.figsize'] = (8, 4)

print("Numpy:", np.__version__)
print("Matplotlib:", plt.matplotlib.__version__)
```

```
Numpy: 2.2.1
Matplotlib: 3.10.8
```

[15]:
```python
def plot_cumulative(rewards, title="Cumulative Reward"):
    plt.figure()
    plt.plot(np.cumsum(rewards))
    plt.title(title)
    plt.xlabel("Step")
    plt.ylabel("Cumulative Reward")
    plt.show()
```

2.2 Environment: Stationary Bernoulli Bandits

Ten arms, each with a hidden probability of reward. You won't see the true means during play, but we print them here for **instructor debugging/analysis**. You may comment this out during the competition.

[16]:
```python
def make_stationary_bandit(n_arms=10, seed=42):
    rng = np.random.default_rng(seed)
    true_means = rng.random(n_arms)  # in [0,1)
    return true_means

# Instructor may reveal (comment out in live competition to keep secret)
SEED_ENV = 42  #  Keep this fixed across all students for fairness (Round 1)
TRUE_MEANS = make_stationary_bandit(seed=SEED_ENV)
print("DEBUG - True means (hidden in competition):", np.round(TRUE_MEANS, 3))
```

```
DEBUG - True means (hidden in competition): [0.774 0.439 0.859 0.697 0.094 0.976
0.761 0.786 0.128 0.45 ]
```

2.3 Agent: -Greedy (Fixed or Decaying)

- With probability , explore a random arm.
- Otherwise, exploit the best arm found so far (highest estimated value).
- Estimates updated via **incremental sample average**.

[17]:
```python
def epsilon_greedy(true_means, steps=1000, epsilon=0.1, seed=None):
    rng = np.random.default_rng(seed)
    n_arms = len(true_means)
    Q = np.zeros(n_arms)      # value estimates
    N = np.zeros(n_arms)      # counts
    rewards = np.zeros(steps, dtype=float)
    actions = np.zeros(steps, dtype=int)
    for t in range(steps):
        if rng.random() < epsilon:
            a = rng.integers(0, n_arms)
        else:
            a = int(np.argmax(Q))
        r = 1.0 if rng.random() < true_means[a] else 0.0
        N[a] += 1
        Q[a] += (r - Q[a]) / N[a]  # incremental mean
```

```
        rewards[t] = r
        actions[t] = a
    return rewards, actions, Q, N

def epsilon_greedy_decaying(true_means, steps=1000, eps_start=0.5, eps_end=0.
 ↪05, seed=None):
    rng = np.random.default_rng(seed)
    n_arms = len(true_means)
    Q = np.zeros(n_arms)
    N = np.zeros(n_arms)
    rewards = np.zeros(steps, dtype=float)
    actions = np.zeros(steps, dtype=int)
    for t in range(steps):
        # Linear decay
        epsilon = eps_end + (eps_start - eps_end) * max(0, (steps - 1 - t)) /␣
 ↪max(1, steps - 1)
        if rng.random() < epsilon:
            a = rng.integers(0, n_arms)
        else:
            a = int(np.argmax(Q))
        r = 1.0 if rng.random() < true_means[a] else 0.0
        N[a] += 1
        Q[a] += (r - Q[a]) / N[a]
        rewards[t] = r
        actions[t] = a
    return rewards, actions, Q, N
```

3 Round 1 — Stationary Casino (Competition)

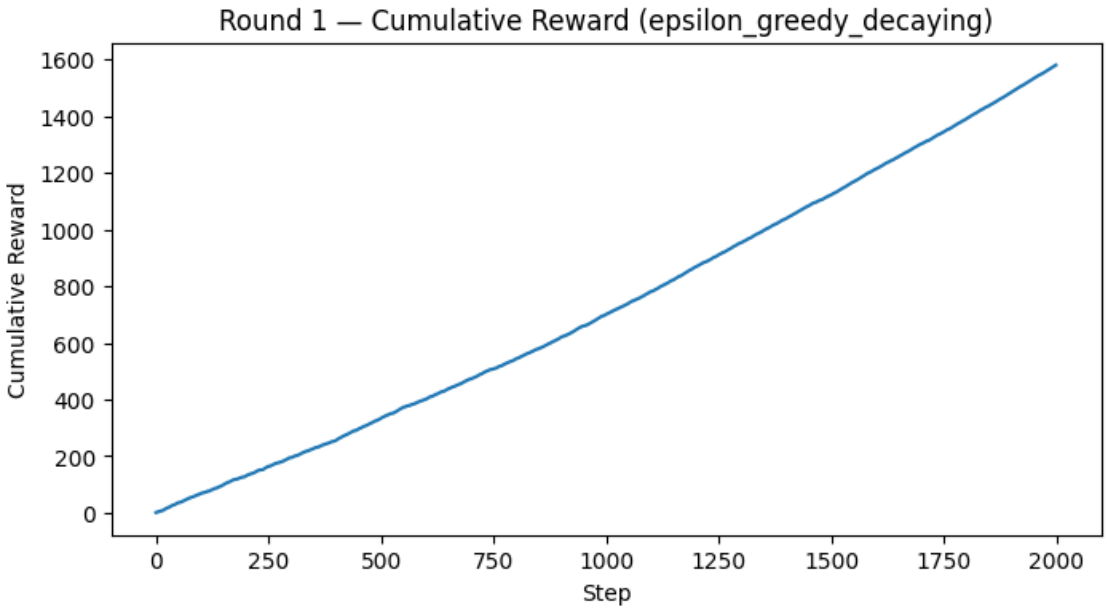
Instructions 1. Set your **NAME** and **STRATEGY**. 2. Choose **steps** and (or decaying parameters). 3. Run the simulation cell. 4. Submit to the local leaderboard (next cell).

> Everyone must use the same **SEED_ENV** to ensure the environment is identical. You can set your **agent seed** for reproducibility.

[18]:
```
# === YOUR SETTINGS (edit) ===
NAME = "Ali Cihan Ozdemir"
STRATEGY = "epsilon_greedy_decaying"  # options: "epsilon_greedy" or␣
 ↪"epsilon_greedy_decaying"
STEPS = 2000

# For fixed
EPSILON = 0.1

# For decaying
EPS_START = 1.0
EPS_END = 0.01
```

```python
# Agent RNG seed (can be None for randomness)
SEED_AGENT = 123

# === RUN ===
if STRATEGY == "epsilon_greedy":
    rewards, actions, Q, N = epsilon_greedy(TRUE_MEANS, steps=STEPS,␣
 ↪epsilon=EPSILON, seed=SEED_AGENT)
    strat_desc = f"fixed_eps={EPSILON}"
elif STRATEGY == "epsilon_greedy_decaying":
    rewards, actions, Q, N = epsilon_greedy_decaying(TRUE_MEANS, steps=STEPS,␣
 ↪eps_start=EPS_START, eps_end=EPS_END, seed=SEED_AGENT)
    strat_desc = f"decay_eps={EPS_START}->{EPS_END}"
else:
    raise ValueError("Unknown STRATEGY setting")

total = float(np.sum(rewards))
best_arm_est = int(np.argmax(Q))
print(f"""
Player: {NAME}
Strategy: {STRATEGY} ({strat_desc})
Steps: {STEPS}
Total Reward: {total:.0f}
Estimated Best Arm: {best_arm_est}
Estimated Q: {np.round(Q,3)}
Counts N: {N.astype(int)}
""")
plot_cumulative(rewards, title=f"Round 1 - Cumulative Reward ({STRATEGY})")
```

```
Player: Ali Cihan Ozdemir
Strategy: epsilon_greedy_decaying (decay_eps=1.0->0.01)
Steps: 2000
Total Reward: 1581
Estimated Best Arm: 5
Estimated Q: [0.789 0.514 0.829 0.685 0.092 0.978 0.818 0.835 0.135 0.495]
Counts N: [ 109  111  105   92  109 1082   88  103  104   97]
```



3.0.1 Submit to Leaderboard (Round 1)

This writes your result to a local CSV (`submissions_round1.csv`) in the current folder. The instructor can collect these files or run the next cell to view a local leaderboard.

[19]:
```
import csv

lb_path = Path("submissions_round1.csv")
lb_exists = lb_path.exists()

row = {
    "timestamp": datetime.utcnow().isoformat(),
    "name": NAME,
    "strategy": STRATEGY,
    "details": strat_desc,
    "steps": STEPS,
    "seed_env": SEED_ENV,
    "seed_agent": SEED_AGENT,
    "total_reward": int(np.sum(rewards))
}

fieldnames =␣
 ↪["timestamp","name","strategy","details","steps","seed_env","seed_agent","total_reward"]

with open(lb_path, "a", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    if not lb_exists:
```

```
        writer.writeheader()
    writer.writerow(row)

print("Submitted to", lb_path.resolve())
```

```
Submitted to /Users/alicihanozdemir/Documents/SpringClassAssignments/CSCN8020/Mu
ltiArmedBandit_Workshop/submissions_round1.csv
```

```
/var/folders/2n/7s2d51jn14q49vthc8fnj1nm0000gn/T/ipykernel_76449/2191877043.py:7
: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  "timestamp": datetime.utcnow().isoformat(),
```

[20]:
```
import pandas as pd
from IPython.display import display

lb_path = Path("submissions_round1.csv")

if lb_path.exists():
    df = pd.read_csv(lb_path)
    df_sorted = df.sort_values("total_reward", ascending=False).
 ↪reset_index(drop=True)
    print("  Round 1 Leaderboard (sorted by total reward):")
    display(df_sorted)
else:
    print("No submissions yet. Run the previous cell to submit your score.")
```

```
  Round 1 Leaderboard (sorted by total reward):
```

	timestamp	name	strategy	details	steps	seed_env	seed_agent	total_reward
0	2026-06-26T15:02:54.577873	Ali Cihan Ozdemir	epsilon_greedy_decaying	decay_eps=1.0->0.01	2000	42	123	1581
1	2026-06-26T15:24:13.858731	Ali Cihan Ozdemir	epsilon_greedy_decaying	decay_eps=1.0->0.01	2000	42	123	1581
2	2026-06-26T15:26:21.292431	Ali Cihan Ozdemir	epsilon_greedy_decaying	decay_eps=1.0->0.01	2000	42	123	1581
3	2026-06-26T15:40:52.910825	Ali Cihan Ozdemir	epsilon_greedy_decaying	decay_eps=1.0->0.01	2000	42	123	1581

3.1 Step 6 — Reflect & Discuss

1. Strategy used and why: - **Round 1:** Decaying ϵ-greedy (1.0 $\rightarrow$ 0.01). *Why:* Stationary target. Heavy initial exploration guarantees finding the best arm; near-pure late exploitation maximizes reward. - **Round 2:** Fixed ϵ (0.15) & Constant α (0.15). *Why:* Drifting target. Constant α acts as an Exponential Moving Average (EMA) to "forget" old data and prioritize recent rewards.

2. ϵ influence: - **High ϵ:** High exploration, wastes steps on suboptimal arms. - **Low ϵ:** High exploitation, risks premature convergence to local optima. - Decaying ϵ optimizes this balance dynamically.

3. Getting "stuck" & prevention: - Agents get stuck if ϵ decays to 0 too rapidly. - **Prevention:** Use a non-zero baseline ϵ (e.g., 0.01) or employ Optimistic Initial Values.

4. Changes for more steps: - **Round 1:** Slow down the decay rate proportionally to the horizon. - **Round 2:** No changes. Constant parameters inherently handle infinite horizons.

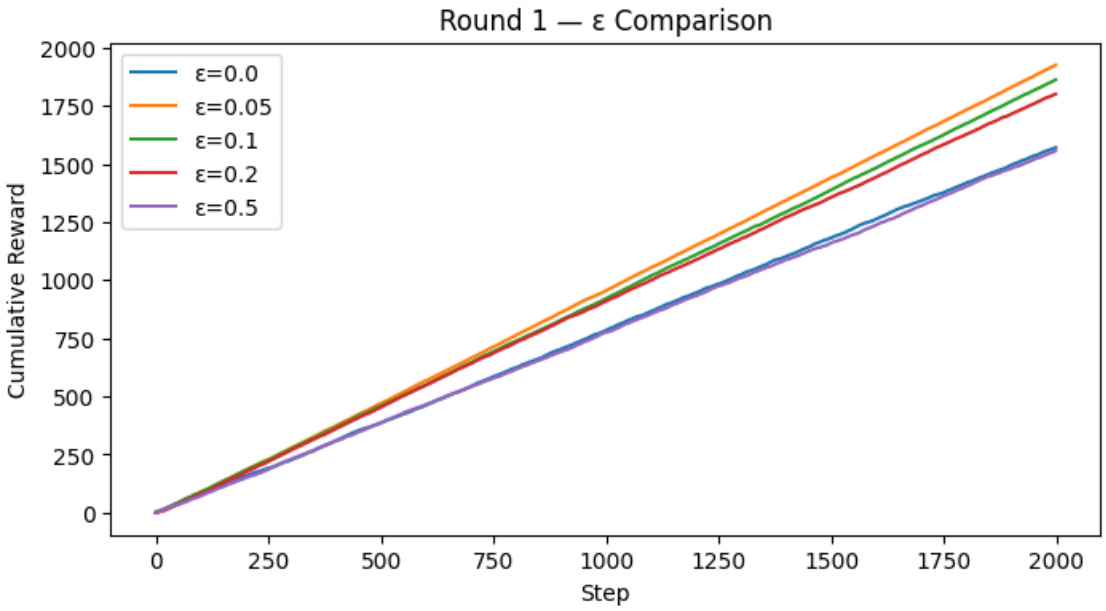
5. Randomness vs Design: - **Design > Randomness.** Random seeds set the baseline, but intelligent algorithm design (decay schedules, recency weighting) mathematically secures higher expected returns.

6. Real-world Transfer: - **Stationary:** A/B Testing (e.g., fixed UI layout conversions). - **Non-Stationary:** Recommender Systems (user preferences drift constantly).

3.2 Experiment: Compare Different Values (Optional)

Run multiple settings to *see* the exploration trade-off.

[21]:
```
eps_list = [0.0, 0.05, 0.1, 0.2, 0.5]
curves = {}
for eps in eps_list:
    r, _, _, _ = epsilon_greedy(TRUE_MEANS, steps=STEPS, epsilon=eps,␣
 ↪seed=SEED_AGENT)
    curves[eps] = np.cumsum(r)

plt.figure()
for eps, curve in curves.items():
    plt.plot(curve, label=f" ={eps}")
plt.legend()
plt.xlabel("Step")
plt.ylabel("Cumulative Reward")
plt.title("Round 1 -  Comparison")
plt.show()
```



4 Round 2 — Non-Stationary Casino (Competition)

Twist: The slot machines drift over time. Fixed exploitation can fail; adaptive exploration helps.

Two common adaptations: - Keep from decaying too low (retain exploration). - Use a **constant step size** (exponential moving average) to weight recent rewards more.

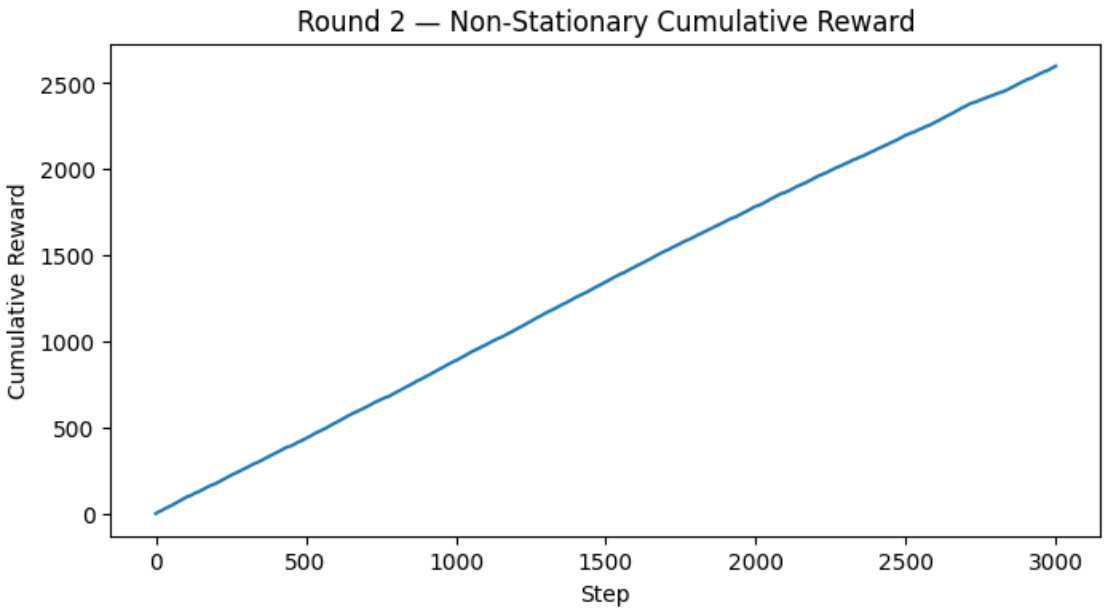
[22]:
```
def nonstationary_means(n_arms=10, seed=2025):
    # Initialize random means
    rng = np.random.default_rng(seed)
    return rng.random(n_arms)

def step_drift(means, drift_scale=0.01, rng=None):
    if rng is None:
        rng = np.random.default_rng()
    means = means + rng.normal(0, drift_scale, size=means.shape)
    return np.clip(means, 0.0, 1.0)

def epsilon_greedy_constant_alpha(steps=2000, n_arms=10, eps=0.1, alpha=0.1,␣
 ↪seed_env=7, seed_agent=None, drift_scale=0.01):
    # Non-stationary env with drifting means
    rng_env = np.random.default_rng(seed_env)
    rng_agent = np.random.default_rng(seed_agent)
    means = rng_env.random(n_arms)
    Q = np.zeros(n_arms)
    rewards = np.zeros(steps, dtype=float)
```

```
    actions = np.zeros(steps, dtype=int)
    for t in range(steps):
        # choose action
        if rng_agent.random() < eps:
            a = rng_agent.integers(0, n_arms)
        else:
            a = int(np.argmax(Q))
        # reward from current means
        r = 1.0 if rng_env.random() < means[a] else 0.0
        # constant step-size update (EMA)
        Q[a] = Q[a] + alpha * (r - Q[a])
        rewards[t] = r
        actions[t] = a
        # drift environment
        means = step_drift(means, drift_scale=drift_scale, rng=rng_env)
    return rewards, actions, Q
```

[23]:
```
# === YOUR SETTINGS (edit) ===
NAME_R2 = "Ali Cihan Ozdemir"
STEPS_R2 = 3000
EPS_R2 = 0.15      # keep some exploration alive
ALPHA_R2 = 0.15    # constant step size for non-stationarity
SEED_ENV_R2 = 2025  # shared across class
SEED_AGENT_R2 = 999
DRIFT_SCALE = 0.01  # magnitude of mean drift per step

# === RUN ===
rewards_r2, actions_r2, Q_r2 = epsilon_greedy_constant_alpha(
    steps=STEPS_R2, n_arms=10, eps=EPS_R2, alpha=ALPHA_R2,
    seed_env=SEED_ENV_R2, seed_agent=SEED_AGENT_R2, drift_scale=DRIFT_SCALE
)

total_r2 = int(np.sum(rewards_r2))
print(f"""
[Round 2]
Player: {NAME_R2}
Strategy: epsilon_greedy + constant_alpha (eps={EPS_R2}, alpha={ALPHA_R2})
Steps: {STEPS_R2}
Total Reward: {total_r2}
""")
plot_cumulative(rewards_r2, title="Round 2 - Non-Stationary Cumulative Reward")
```

```
[Round 2]
Player: Ali Cihan Ozdemir
Strategy: epsilon_greedy + constant_alpha (eps=0.15, alpha=0.15)
Steps: 3000
```

```
Total Reward: 2599
```



[24]:
```python
import csv
lb2_path = Path("submissions_round2.csv")
lb2_exists = lb2_path.exists()

row2 = {
    "timestamp": datetime.utcnow().isoformat(),
    "name": NAME_R2,
    "strategy": f"eps={EPS_R2}, alpha={ALPHA_R2}",
    "steps": STEPS_R2,
    "seed_env": SEED_ENV_R2,
    "seed_agent": SEED_AGENT_R2,
    "drift_scale": DRIFT_SCALE,
    "total_reward": total_r2
}

fieldnames2 =␣
 ↪["timestamp","name","strategy","steps","seed_env","seed_agent","drift_scale","total_reward"]

with open(lb2_path, "a", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames2)
    if not lb2_exists:
        writer.writeheader()
    writer.writerow(row2)
```

```
print("Submitted to", lb2_path.resolve())
```

```
Submitted to /Users/alicihanozdemir/Documents/SpringClassAssignments/CSCN8020/Mu
ltiArmedBandit_Workshop/submissions_round2.csv

/var/folders/2n/7s2d51jn14q49vthc8fnj1nm0000gn/T/ipykernel_76449/2415904048.py:6
: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes
in UTC: datetime.datetime.now(datetime.UTC).
  "timestamp": datetime.utcnow().isoformat(),
```

[25]:
```
import pandas as pd
from IPython.display import display

lb_path = Path("submissions_round2.csv")

if lb_path.exists():
    df = pd.read_csv(lb_path)
    df_sorted = df.sort_values("total_reward", ascending=False).
 ↪reset_index(drop=True)
    print("  Round 1 Leaderboard (sorted by total reward):")
    display(df_sorted)
else:
    print("No submissions yet. Run the previous cell to submit your score.")
```

```
  Round 1 Leaderboard (sorted by total reward):
```

	timestamp	name	strategy	steps
0	2026-06-26T15:02:55.501027	Ali Cihan Ozdemir	eps=0.15, alpha=0.15	3000
1	2026-06-26T15:24:14.416808	Ali Cihan Ozdemir	eps=0.15, alpha=0.15	3000
2	2026-06-26T15:26:22.043903	Ali Cihan Ozdemir	eps=0.15, alpha=0.15	3000
3	2026-06-26T15:40:53.076347	Ali Cihan Ozdemir	eps=0.15, alpha=0.15	3000

	seed_env	seed_agent	drift_scale	total_reward
0	2025	999	0.01	2599
1	2025	999	0.01	2599
2	2025	999	0.01	2599
3	2025	999	0.01	2599

4.1 Bonus (Optional): Compete with UCB or Thompson Sampling

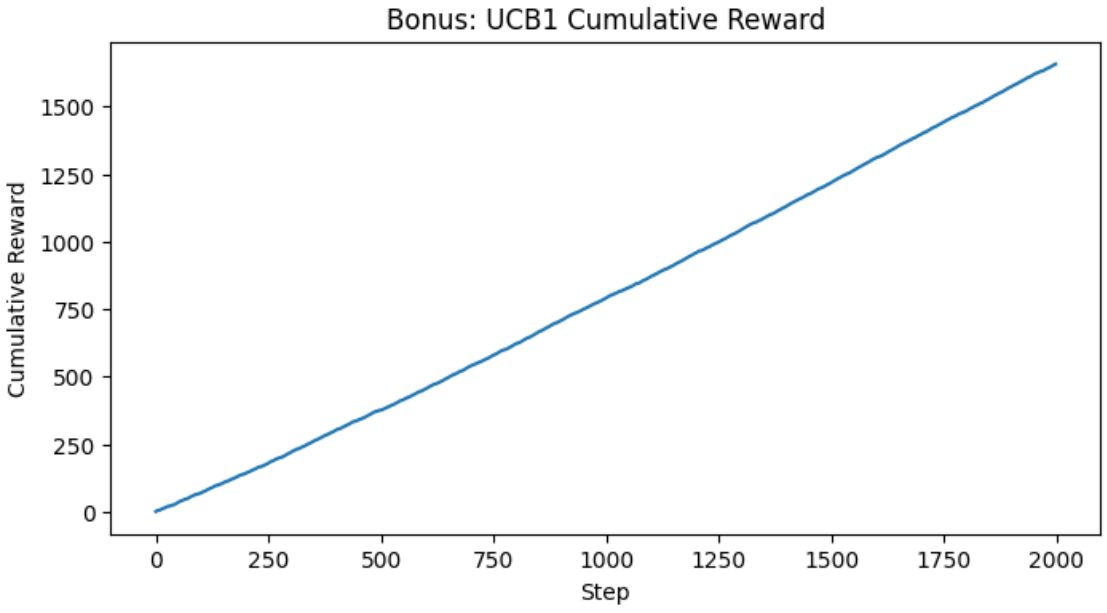
If allowed by the instructor, try implementing: - **UCB1 (Upper Confidence Bound):** add an optimism bonus to less-tried arms. - **Thompson Sampling:** maintain Beta posteriors for each arm and sample to choose.

Keep Round 1/2 seeds the same for comparability.

[26]:
```
def ucb1_bandit(true_means, steps=2000, c=2.0, seed=123):
    rng = np.random.default_rng(seed)
```

```
    n_arms = len(true_means)
    Q = np.zeros(n_arms)
    N = np.zeros(n_arms)
    rewards = np.zeros(steps, dtype=float)

    for t in range(steps):
        if t < n_arms: # Play each arm once
            a = t
        else:
            ucb_values = Q + c * np.sqrt(np.log(t + 1) / N)
            a = int(np.argmax(ucb_values))

        r = 1.0 if rng.random() < true_means[a] else 0.0
        N[a] += 1
        Q[a] += (r - Q[a]) / N[a]
        rewards[t] = r

    return rewards, Q


rewards_ucb, Q_ucb = ucb1_bandit(TRUE_MEANS, steps=2000, c=2.0)
print("Bonus UCB1 completed by Ali Cihan Ozdemir")
print(f"UCB1 Total Reward: {int(np.sum(rewards_ucb))}")
plot_cumulative(rewards_ucb, title="Bonus: UCB1 Cumulative Reward")
```

```
Bonus UCB1 completed by Ali Cihan Ozdemir
UCB1 Total Reward: 1657
```



4.2 Talking Points

- **The Core Dilemma:** RL agents must continuously balance exploring unknown states to find better policies vs. exploiting known states to maximize immediate reward.
- **Stationarity:** When the environment dynamics are fixed, algorithms that decay exploration over time (like decaying ϵ-greedy) mathematically converge to the optimal policy.
- **Non-Stationarity:** Real-world problems (like stock markets or user recommendations) drift. Using a constant step-size (α) is crucial to override stale historical data with recent observations.

4.3 Conclusion

This workshop practically demonstrated how different RL algorithms adapt to environmental stability. While decaying ϵ-greedy dominates in stationary settings by systematically eliminating suboptimal choices, it fails entirely in non-stationary environments. By introducing a constant step-size (α) and maintaining a baseline exploration rate, the agent successfully learned to track drifting probabilities, proving that algorithmic flexibility is essential for dynamic real-world applications.